

RAPPORT TECHNIQUE

Système de Gestion de Flotte de Véhicules

Semaine 2

Architecture, Infrastructure & Conteneurisation

Informations du projet

Auteurs :	Kherbachi Rima & Mouhoubi Leiticia
Formation :	Master 1 GIL
Université :	Université de Rouen Normandie
Encadrants :	Lydia et Luc
Année :	2025 – 2026
Semaines documentées :	1 et 2

Mars 2026

Chapitre 1

Introduction

Ce rapport présente le travail réalisé durant les deux premières semaines du projet de Gestion de Flotte de Véhicules dans le cadre du Master 1 GIL de l'Université de Rouen Normandie. L'intégralité du code source (scripts de déploiement, manifestes Kubernetes et code des microservices) est jointe en annexe de ce document et disponible sur notre dépôt de gestion de versions.

- **Semaine 2** : Mise en place de l'infrastructure complète de développement et de déploiement (Docker, Kubernetes, CI/CD et stack d'observabilité).

Objectifs du Sprint

- Concevoir une architecture microservices robuste et évolutive.
- Mettre en place des stratégies de conteneurisation optimisées et sécurisées (multi-stage builds).
- Définir et déployer l'infrastructure orchestrée sur Kubernetes (via Minikube).
- Mettre en place la stack d'observabilité (Prometheus, Grafana, Loki, Jaeger, Open-Telemetry).
- Automatiser le cycle de vie logiciel via un pipeline CI/CD (GitHub Actions).

Chapitre 2

Semaine 1 — Conception et Architecture Decision Records

2.1 Vue d'ensemble de l'architecture

Après analyse des besoins du système de gestion de flotte, nous avons identifié cinq domaines métiers distincts et indépendants. Chaque domaine correspond à un microservice dédié, avec sa propre technologie et sa propre base de données.

Ce choix d'architecture polyglotte (plusieurs langages selon les besoins) nous a permis d'utiliser l'outil le plus adapté à chaque contexte : Java/Spring Boot pour la robustesse du service central, Python/FastAPI pour la rapidité de développement des services événementiels, Node.js/NestJS pour les I/O intensifs, et Node.js/gRPC pour le streaming GPS temps réel.

secondary			
vehicule-service	Spring Boot (Java)	8080	CRUD véhicules, disponibilité, historique
conducteur-service	NestJS (Node.js)	3000	Profils conducteurs, permis, assignations
maintenance-service	FastAPI (Python)	8000	Planification et suivi des interventions
evenement-service	FastAPI (Python)	8000	Alertes, notifications, événements Kafka
localisation-service	Node.js gRPC	50051	Tracking GPS temps réel

TABLE 2.1 – Les 5 microservices du système

2.2 Architecture Decision Records (ADR)

Un ADR (Architecture Decision Record) est un document court qui trace une décision architecturale importante : le contexte qui a mené à cette décision, la décision elle-même, et ses conséquences. Nous avons documenté 6 ADR pour les choix les plus structurants du projet.

2.2.1 ADR-001 : Architecture Microservices

ADR-001 — Architecture Microservices | Statut : Accepté

Contexte : Le système de gestion de flotte couvre plusieurs domaines métiers distincts — véhicules, conducteurs, maintenance, localisation, événements. Une architecture monolithique rendrait difficile l'évolution indépendante de chaque domaine et créerait des risques en cas de panne.

Décision : Nous adoptons une architecture microservices avec 5 services indépendants. Chaque service est responsable d'un seul domaine métier, possède sa propre base de données, et communique avec les autres via des messages asynchrones (Kafka) plutôt que des appels directs.

Conséquences :

- Chaque service peut être développé, testé et déployé indépendamment
- La défaillance d'un service n'entraîne pas la panne de l'ensemble du système
- La complexité opérationnelle est plus élevée, nécessitant une infrastructure Kubernetes
- Une observabilité distribuée est indispensable pour suivre les requêtes entre services

2.2.2 ADR-002 : Architecture Polyglotte

ADR-002 — Architecture Polyglotte | Statut : Accepté

Contexte : Les différents services ont des contraintes techniques distinctes. Le service véhicule nécessite robustesse et transactions fiables. Le service localisation doit gérer un flux de données GPS en temps réel. Les services maintenance et événement sont principalement des traitements de données.

Décision : Nous utilisons la technologie la plus adaptée pour chaque service : Java/Spring Boot pour vehicule-service (maturité, JPA, transactions), Node.js/NestJS pour conducteur-service (architecture modulaire, TypeScript), Python/FastAPI pour maintenance-service et evenement-service (simplicité, rapidité de développement), Node.js/gRPC pour localisation-service (streaming binaire efficace).

Conséquences :

- Chaque service est optimisé pour son cas d'usage spécifique
- Les Dockerfiles sont différents pour chaque service selon le langage
- L'équipe doit maîtriser plusieurs technologies

2.2.3 ADR-003 : Apache Kafka comme bus de messages

ADR-003 — Apache Kafka | Statut : Accepté

Contexte : Les services doivent pouvoir communiquer entre eux sans être directement couplés. Par exemple, quand un véhicule change d'état, le service de maintenance et le service d'événements doivent en être informés, sans que *vehicule-service* connaisse l'adresse de ces services.

Décision : Nous utilisons Apache Kafka comme bus de messages événementiel. Kafka est déployé via l'opérateur Strimzi sur Kubernetes. Quatre topics sont définis : *fleet.vehicules.events*, *fleet.maintenance.events*, *fleet.geofencing.alerts* et *fleet.notifications*.

Conséquences :

- Les services communiquent de manière asynchrone et découplée
- Les messages sont persistés — un service peut rejouer les événements en cas de panne
- Bitnami Kafka étant devenu payant depuis août 2025, nous avons opté pour Strimzi (voir ADR-005)

2.2.4 ADR-004 : Database per Service

ADR-004 — Database per Service | Statut : Accepté

Contexte : Dans une architecture microservices, partager une seule base de données entre plusieurs services créerait un couplage fort au niveau des données. Une modification du schéma d'un service pourrait casser les autres.

Décision : Chaque service possède sa propre instance de base de données isolée. Nous utilisons 4 instances PostgreSQL (*vehicules_db*, *conducteurs_db*, *maintenance_db*, *evenements_db*) et une instance TimescaleDB pour les données GPS time-series du service de localisation. Toutes sont déployées via Helm Bitnami.

Conséquences :

- Chaque service peut faire évoluer son schéma sans impacter les autres
- Les données de chaque domaine sont totalement isolées
- Plus de ressources sont consommées — ce compromis est acceptable en développement

2.2.5 ADR-005 : Strimzi pour Kafka

ADR-005 — Strimzi pour Kafka | Statut : Accepté

Contexte : Lors de nos tests, le chart Helm Bitnami pour Kafka générait une erreur `Init:Error` car les images Docker Bitnami sont devenues payantes depuis août 2025. Tous les pods Kafka restaient bloqués en état d'initialisation.

Décision : Nous remplaçons Bitnami par Strimzi, un opérateur Kubernetes open-source pour Apache Kafka, maintenu par la CNCF. L'opérateur est déployé via Helm, puis le cluster Kafka est déclaré en YAML via une Custom Resource Definition (CRD). La version utilisée est `strimzi-kafka-operator-0.51.0`.

Conséquences :

- Solution 100% gratuite et open-source, sans restriction d'accès
- Meilleure intégration Kubernetes : l'opérateur surveille et gère le cycle de vie de Kafka automatiquement
- C'est le standard industriel utilisé en production par Red Hat, IBM et d'autres grandes entreprises
- Le déploiement en deux étapes (Helm + CRD YAML) est légèrement plus complexe mais bien documenté

2.2.6 ADR-006 : Observabilité avec OpenTelemetry

ADR-006 — Observabilité OpenTelemetry | Statut : Accepté

Contexte : Dans une architecture distribuée avec 5 services indépendants, il est difficile de diagnostiquer les problèmes sans visibilité sur ce qui se passe dans chaque service et entre les services. Nous avons besoin de savoir : quelles métriques consomme chaque service ? Où se trouvent les goulots d'étranglement ? Quels logs correspondent à quelle requête ?

Décision : Nous adoptons la stack d'observabilité complète basée sur OpenTelemetry, couvrant les 3 piliers : **Prometheus** + **Grafana** pour les métriques, **Loki** pour l'agrégation des logs, **Jaeger** pour le tracing distribué des requêtes, et **OpenTelemetry Collector** comme point central de collecte.

Conséquences :

- Visibilité complète sur le comportement du système en production
- Possibilité de créer des dashboards Grafana pour les métriques métier
- Conformité aux standards cloud-native et aux bonnes pratiques de l'industrie

Chapitre 3

Semaine 2 — Infrastructure et Déploiement

3.1 Dockerfiles multi-stage

Pour chaque microservice, nous avons créé un Dockerfile utilisant la technique du **multi-stage build**. Cette approche consiste à séparer le processus en deux étapes distinctes dans un même fichier :

- **Étape 1 — Build** : Une image complète avec tous les outils de compilation (Maven pour Java, npm pour Node.js, pip pour Python) est utilisée pour compiler le code source et produire les artefacts.
- **Étape 2 — Production** : Une image beaucoup plus légère, sans les outils de build, récupère uniquement les artefacts compilés. Cela réduit considérablement la taille finale de l'image.

Un aspect important de sécurité : tous nos Dockerfiles créent un utilisateur non-root dédié (spring, nodejs, appuser) et basculent sur cet utilisateur avant de lancer l'application. Cela suit le principe du moindre privilège — si un conteneur est compromis, l'attaquant n'a pas les droits root sur le système.

3.1.1 vehicule-service (Spring Boot / Java)

Pour ce service Java, l'étape de build utilise l'image `maven:3.9-eclipse-temurin-17-alpine`. Nous copions d'abord uniquement le fichier `pom.xml` pour télécharger les dépendances Maven en cache, puis le code source pour la compilation. Cette optimisation évite de retélécharger les dépendances à chaque changement de code.

L'image de production utilise `eclipse-temurin:17-jre-alpine` — le JRE (Java Runtime Environment) au lieu du JDK complet, ce qui divise la taille de l'image par 3. Le fichier `.jar` compilé est copié et l'application est démarrée avec `java -jar`.

3.1.2 conducteur-service (NestJS / Node.js)

Pour ce service TypeScript/NestJS, le build compile le TypeScript vers JavaScript (`npm run build`). L'image de production installe uniquement les dépendances de production (`npm ci --omit=dev`), excluant tous les outils de développement. Le répertoire `dist/` contenant le JavaScript compilé est copié dans l'image finale.

3.1.3 maintenance-service & evenement-service (FastAPI / Python)

Ces deux services Python suivent le même pattern. L'étape de build installe les dépendances Python dans le répertoire de l'utilisateur (`pip install -user`). L'image de production copie ces dépendances déjà installées, évitant de les réinstaller. La variable `PYTHONUNBUFFERED=1` est définie pour que les logs s'affichent immédiatement dans Docker sans mise en tampon. L'application est démarrée avec Uvicorn, le serveur ASGI recommandé pour FastAPI.

3.1.4 localisation-service (Node.js / gRPC)

Ce service suit le même pattern que conducteur-service, avec une particularité importante : les fichiers `.proto` définissant le contrat gRPC ne sont pas compilés par TypeScript. Ils doivent donc être copiés manuellement dans l'image de production aux côtés du code compilé pour que le serveur gRPC puisse les lire au démarrage.

3.2 Docker Compose

Le fichier `docker-compose.yml` a été créé pour permettre de lancer l'ensemble de l'infrastructure localement sans Kubernetes, ce qui facilite le développement et les tests rapides.

Il définit deux catégories de services :

Infrastructure : PostgreSQL (port 5432), TimescaleDB sur le port 5433 pour éviter le conflit avec PostgreSQL, Redis (port 6379), Kafka (port 9092) et Keycloak pour l'authentification (port 9080). Tous ces services partagent le même réseau interne `fleet-network`.

Microservices : Les 5 services métiers sont définis avec leur contexte de build Docker, leurs ports exposés et leurs variables d'environnement. Chaque service dépend des composants d'infrastructure dont il a besoin (`depends_on`), garantissant que la base de données et Kafka démarrent avant le service.

Le lancement se fait en une seule commande :

Listing 3.1 – Lancement de l'environnement de developpement local

```
1 docker compose up -d
```

```

PS C:\Systeme-de-Gestion-de-Flotte-de-Vehicules> docker compose up -d
✓ Container fleet-redis Started 2.0s
✓ Container fleet-timescaledb Started 2.1s
✓ Container fleet-postgres Started 2.4s
✓ Container fleet-kafka Started 2.0s
✓ Container fleet-localisation-service Started 3.0s
✓ Container fleet-keycloak Started 2.8s
✓ Container fleet-evenement-service Started 3.0s
✓ Container fleet-maintenance-service Started 3.4s
✓ Container fleet-conducteur-service Started 2.9s
✓ Container fleet-vehicule-service Started 3.1s
PS C:\Systeme-de-Gestion-de-Flotte-de-Vehicules>

```

FIGURE 3.1 – Les conteneurs du Docker Compose démarrent et opérationnels

3.3 Manifests Kubernetes

Les manifests Kubernetes définissent l'état désiré de notre infrastructure dans le cluster. Nous les avons organisés dans le dossier `k8s/` avec un fichier par ressource pour faciliter la maintenance.

3.3.1 Namespace

Nous avons créé un namespace dédié `fleet-management` pour isoler toutes nos ressources des autres applications qui pourraient tourner sur le même cluster.

3.3.2 Secrets

Les secrets Kubernetes stockent les informations sensibles (mots de passe, URLs de connexion) de manière encodée en base64. Ces secrets sont référencés dans les Deployments via `valueFrom: secretKeyRef`, évitant d'écrire les mots de passe en clair dans les fichiers YAML.

3.3.3 Deployments

Chaque microservice est décrit dans un Deployment Kubernetes configuré avec :

- **Replicas** : 2 replicas pour assurer la haute disponibilité.
- **imagePullPolicy** : **Never** : Indique à Kubernetes d'utiliser les images locales fraîchement compilées.
- **livenessProbe** : Des sondes de santé pour s'assurer que le service est fonctionnel.
- **Resources** : Des limites de ressources CPU et mémoire pour une orchestration optimale.

3.3.4 Services Kubernetes & Ingress

Tous nos Services sont de type `ClusterIP`. L'Ingress est le point d'entrée unique pour les requêtes extérieures. Nous avons configuré l'Ingress avec le hostname `api.fleet.local` et défini les routes adéquates pour chaque service via traefik.

3.3.5 Kafka — Strimzi CRD

Le fichier `kafka.yaml` utilise une Custom Resource Definition (CRD) de Strimzi pour déclarer le cluster Kafka de manière déclarative, laissant l'opérateur se charger de la création technique des pods.

3.4 Commandes de déploiement (Playbook)

La reproductibilité de l'environnement est assurée par un processus de déploiement strict. Voici le manuel des opérations pour instancier l'intégralité du projet depuis un environnement vierge. Les longues commandes ont été formatées sur plusieurs lignes pour une lisibilité optimale.

3.4.1 1. Démarrer Minikube

Initialisation du cluster local et configuration du terminal pour utiliser le daemon Docker de Minikube.

Listing 3.2 – Démarrage de l’environnement Minikube

```
1 minikube start
2 minikube docker-env | Invoke-Expression
```

3.4.2 2. Builder les images Docker

Création des artefacts locaux grâce aux Dockerfiles multi-stage.

Listing 3.3 – Compilation des images métiers

```
1 docker build -t fleet-vehicule-service:latest ./services/vehicule-
  service
2 docker build -t fleet-conducteur-service:latest ./services/
  conducteur-service
3 docker build -t fleet-maintenance-service:latest ./services/
  maintenance-service
4 docker build -t fleet-evenement-service:latest ./services/evenement
  -service
5 docker build -t fleet-localisation-service:latest ./services/
  localisation-service
```

3.4.3 3. Ajouter les repos Helm

Ajout des dépôts nécessaires pour les briques d’infrastructure.

Listing 3.4 – Configuration des dépôts Helm

```
1 helm repo add bitnami https://charts.bitnami.com/bitnami
2 helm repo add strimzi https://strimzi.io/charts/
3 helm repo add prometheus-community https://prometheus-community.
  github.io/helm-charts
4 helm repo add grafana https://grafana.github.io/helm-charts
5 helm repo add open-telemetry https://open-telemetry.github.io/
  opentelemetry-helm-charts
6 helm repo update
```

3.4.4 4. Namespace et secrets

Mise en place des fondations de sécurité du cluster.

Listing 3.5 – Application des configurations de base

```
1 kubectl apply -f k8s/namespace.yaml
2 kubectl apply -f k8s/secrets.yaml
```

3.4.5 5. Infrastructure Helm

Déploiement asynchrone des composants lourds. L'architecture "Database per Service" se reflète ici avec le lancement de 5 bases distinctes.

Listing 3.6 – Déploiement de l'infrastructure via Helm

```
1 # Bases de donnees PostgreSQL (une par service)
2 helm install postgres bitnami/postgresql \
3   --namespace fleet-management \
4   --set auth.username=admin \
5   --set auth.password=adminpassword \
6   --set auth.database=vehicules_db
7
8 helm install postgres-conducteur bitnami/postgresql \
9   --namespace fleet-management \
10  --set auth.username=admin \
11  --set auth.password=adminpassword \
12  --set auth.database=conducteurs_db
13
14 helm install postgres-maintenance bitnami/postgresql \
15   --namespace fleet-management \
16   --set auth.username=admin \
17   --set auth.password=adminpassword \
18   --set auth.database=maintenance_db
19
20 helm install postgres-evenement bitnami/postgresql \
21   --namespace fleet-management \
22   --set auth.username=admin \
23   --set auth.password=adminpassword \
24   --set auth.database=evenements_db
25
26 helm install timescaledb bitnami/postgresql \
27   --namespace fleet-management \
28   --set auth.username=admin \
29   --set auth.password=adminpassword \
30   --set auth.database=localisation_db
31
32 # Redis
33 helm install redis bitnami/redis \
34   --namespace fleet-management \
35   --set auth.enabled=false
36
37 # Kafka via Strimzi (Déploiement de l'opérateur)
38 helm install strimzi-operator strimzi/strimzi-kafka-operator \
39   --namespace fleet-management
40
41 # Observabilite
42 helm install obs prometheus-community/kube-prometheus-stack \
43   --namespace fleet-management
44
45 helm install loki grafana/loki-stack \
```

```
46 --namespace fleet-management
47
48 helm install otel-collector open-telemetry/opentelemetry-collector \
49 --namespace fleet-management \
50 --set mode=deployment \
51 --set image.repository="otel/opentelemetry-collector-contrib"
```

3.4.6 6. Kafka (Cluster via CRD)

Une fois l'opérateur Strimzi démarré (environ 2 minutes de délai), nous lui soumettons notre définition de cluster.

Listing 3.7 – Instanciation du cluster Kafka

```
1 kubectl apply -f k8s/kafka.yaml
```

3.4.7 7. Jaeger (Tracing)

Listing 3.8 – Déploiement du serveur de traçabilité Jaeger

```
1 kubectl run jaeger \
2 --image=jaegertracing/all-in-one:latest \
3 -n fleet-management \
4 --port=16686
5
6 kubectl expose pod jaeger \
7 -n fleet-management \
8 --type=NodePort \
9 --port=16686
```

3.4.8 8. Déploiement des Microservices

Application finale des manifestes gérant notre code métier et l'Ingress.

Listing 3.9 – Lancement applicatif

```
1 kubectl apply -f k8s/vehicule-deployment.yaml
2 kubectl apply -f k8s/vehicule-service.yaml
3 kubectl apply -f k8s/conducteur-deployment.yaml
4 kubectl apply -f k8s/conducteur-service.yaml
5 kubectl apply -f k8s/maintenance-deployment.yaml
6 kubectl apply -f k8s/maintenance-service.yaml
7 kubectl apply -f k8s/evenement-deployment.yaml
8 kubectl apply -f k8s/evenement-service.yaml
9 kubectl apply -f k8s/localisation-deployment.yaml
10 kubectl apply -f k8s/localisation-service.yaml
11 kubectl apply -f k8s/ingress.yaml
```

3.4.9 9. Vérification de l'état du cluster

Listing 3.10 – Commandes de diagnostic

```
1 kubectl get all -n fleet-management
2 helm list -n fleet-management
```

3.4.10 10. Nettoyage de l'environnement (Teardown)

Pour libérer les ressources locales une fois les tests terminés, la désinstallation complète est maîtrisée de façon propre :

Listing 3.11 – Suppression propre du déploiement

```
1 kubectl delete namespace fleet-management
2
3 helm uninstall postgres postgres-conducteur \
4   postgres-maintenance postgres-evenement \
5   timescaledb redis strimzi-operator \
6   obs loki otel-collector \
7   -n fleet-management
```

Chapitre 4

Validation, CI/CD et Résultats

4.1 Intégration et Déploiement Continus (CI/CD)

Le processus a été fiabilisé en automatisant le cycle de vie logiciel via un pipeline CI/CD GitHub Actions. Ce pipeline gère les étapes de construction (Build), de validation et de tests de l'application.

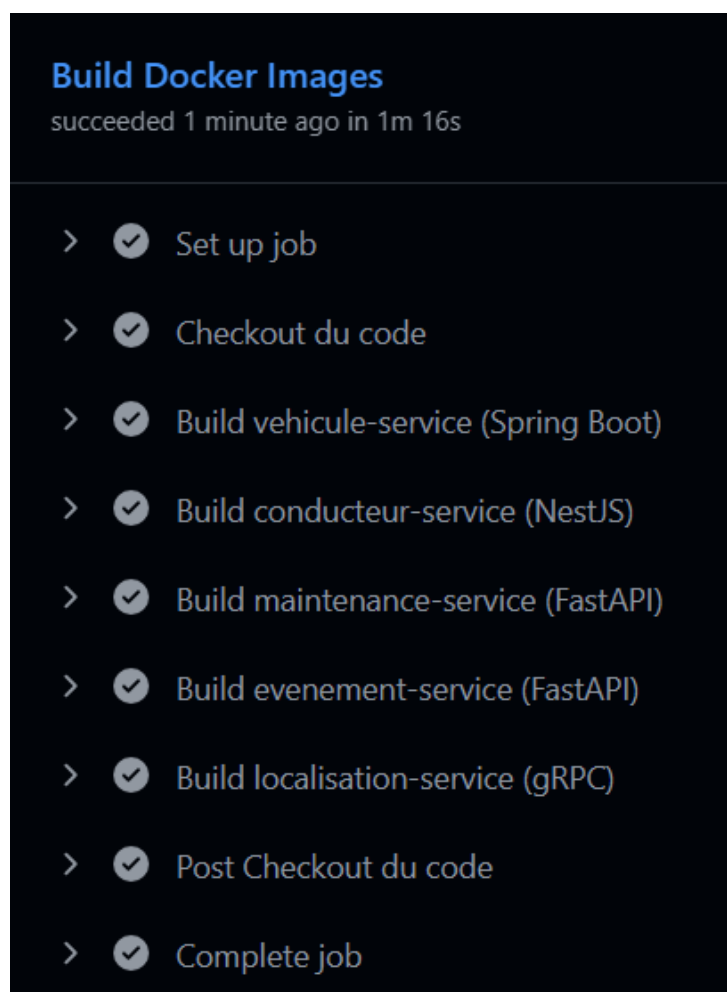


FIGURE 4.1 – Pipeline CI/CD : Exécution du job de construction des images Docker pour les microservices

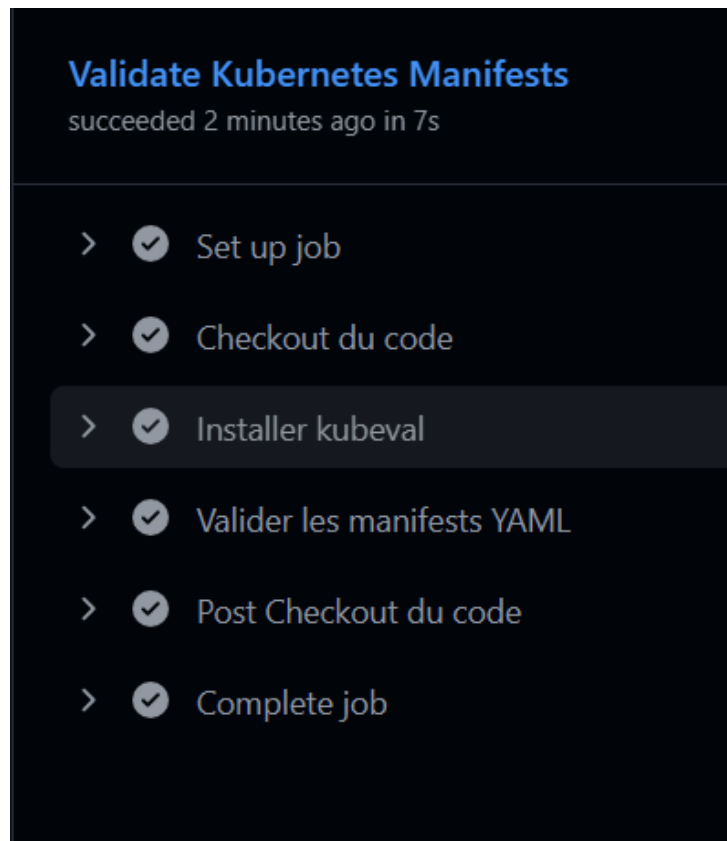


FIGURE 4.2 – Pipeline CI/CD : Exécution du job de validation des manifestes Kubernetes via kubeval

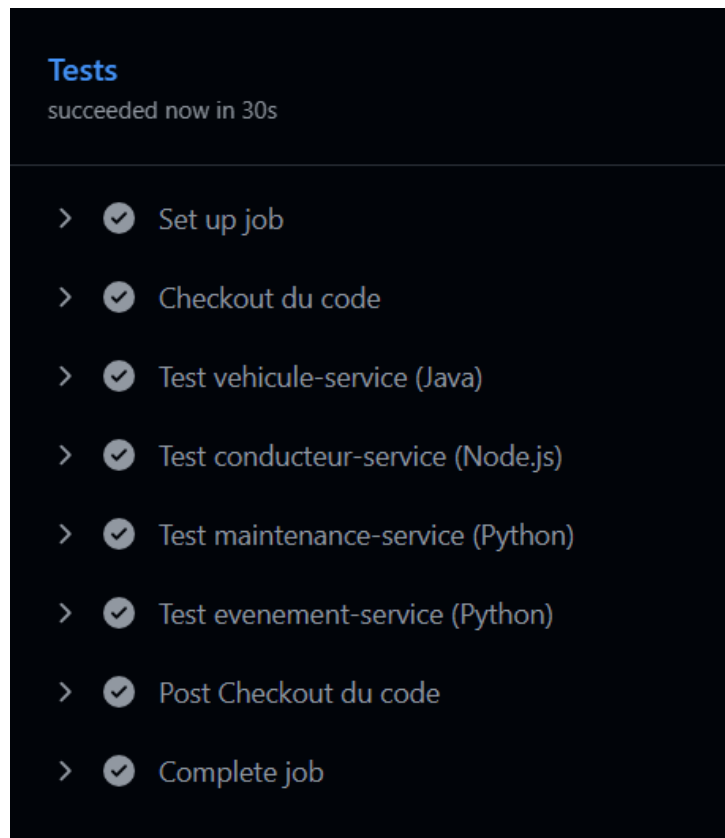


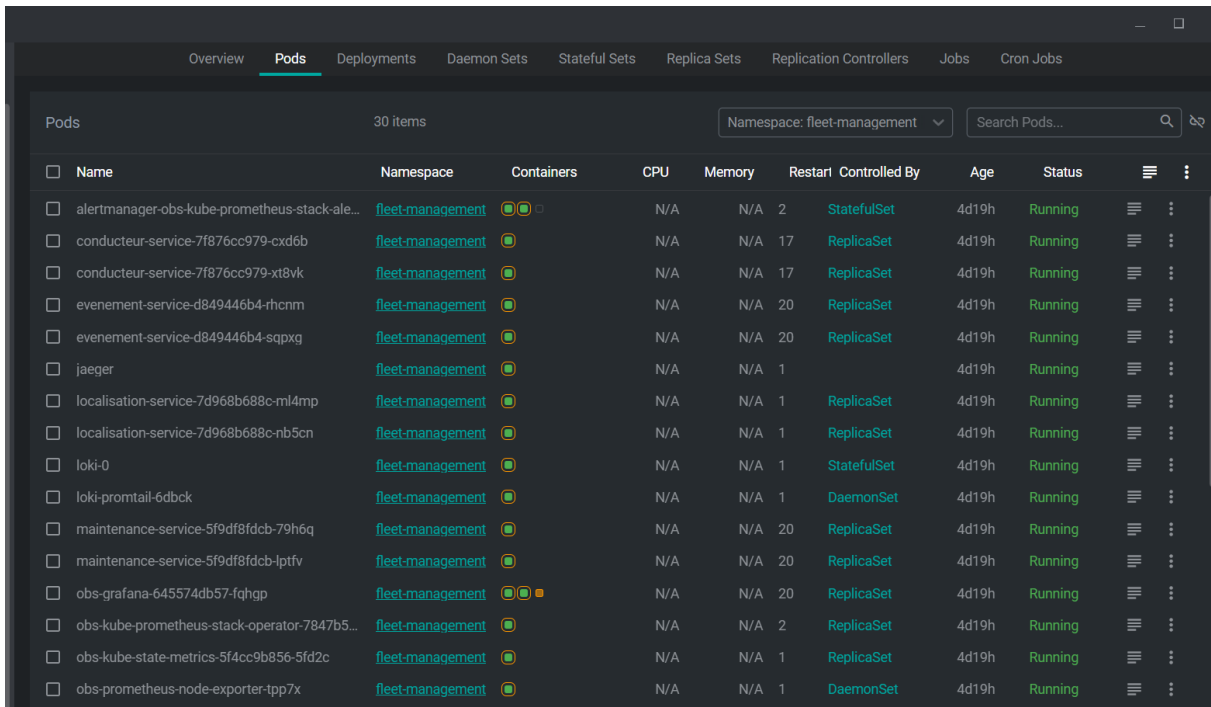
FIGURE 4.3 – Pipeline CI/CD : Exécution du job de tests automatisés par service

4.2 Validation de l'Infrastructure (FreeLens)

Le déploiement intégral a été validé grâce à des outils de diagnostic comme FreeLens. L'état du cluster Kubernetes post-déploiement montre la bonne santé de l'infrastructure avec 30 pods actifs en état *Running*, 10 Deployments disponibles, et 10 Stateful Sets pour la gestion des composants persistants (bases de données). L'architecture polyglotte et distribuée est ainsi pleinement fonctionnelle sur son environnement orchestré.

Les captures d'écran suivantes montrent l'état du cluster Kubernetes dans FreeLens après le déploiement complet. FreeLens est une interface graphique open-source pour Kubernetes qui permet de visualiser en temps réel l'état de toutes les ressources du cluster.

4.2.1 Vue Pods — Namespace fleet-management

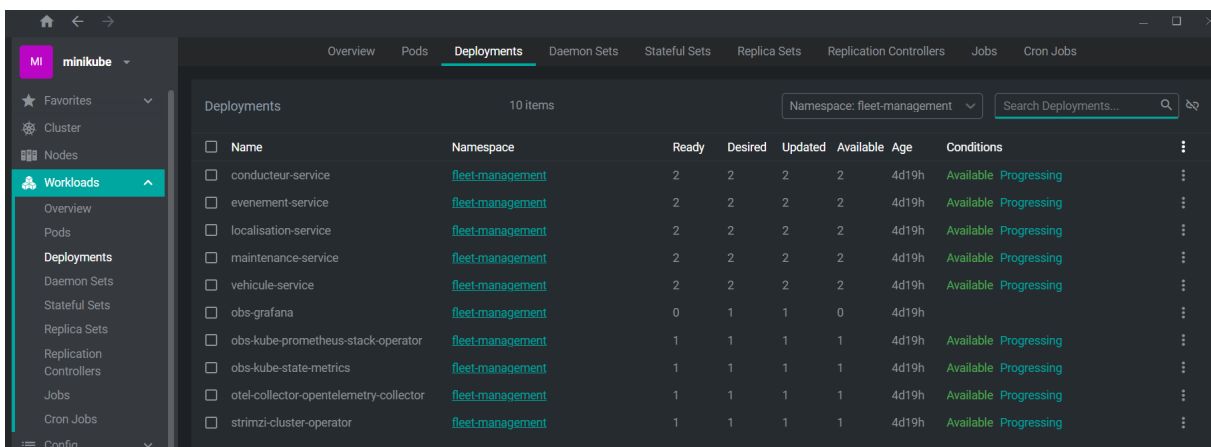


The screenshot shows the 'Pods' view in the Kubernetes dashboard for the 'fleet-management' namespace. It displays a list of 30 pods, all in a 'Running' state. The table includes columns for Name, Namespace, Containers, CPU, Memory, Restart, Controlled By, Age, and Status. The pods are managed by various controllers like StatefulSet, ReplicaSet, and DaemonSet.

Name	Namespace	Containers	CPU	Memory	Restart	Controlled By	Age	Status
alertmanager-obs-kube-prometheus-stack-ale...	fleet-management	2	N/A	N/A	2	StatefulSet	4d19h	Running
conducteur-service-7f876cc979-cxd6b	fleet-management	1	N/A	N/A	17	ReplicaSet	4d19h	Running
conducteur-service-7f876cc979-xt8vk	fleet-management	1	N/A	N/A	17	ReplicaSet	4d19h	Running
evenement-service-d849446b4-rhcnm	fleet-management	1	N/A	N/A	20	ReplicaSet	4d19h	Running
evenement-service-d849446b4-sqpxg	fleet-management	1	N/A	N/A	20	ReplicaSet	4d19h	Running
jaeger	fleet-management	1	N/A	N/A	1		4d19h	Running
localisation-service-7d968b688c-ml4mp	fleet-management	1	N/A	N/A	1	ReplicaSet	4d19h	Running
localisation-service-7d968b688c-nb5cn	fleet-management	1	N/A	N/A	1	ReplicaSet	4d19h	Running
loki-0	fleet-management	1	N/A	N/A	1	StatefulSet	4d19h	Running
loki-promtail-6dbck	fleet-management	1	N/A	N/A	1	DaemonSet	4d19h	Running
maintenance-service-5f9df8fdb-79h6q	fleet-management	1	N/A	N/A	20	ReplicaSet	4d19h	Running
maintenance-service-5f9df8fdb-lptfv	fleet-management	1	N/A	N/A	20	ReplicaSet	4d19h	Running
obs-grafana-645574db57-fqhgp	fleet-management	2	N/A	N/A	20	ReplicaSet	4d19h	Running
obs-kube-prometheus-stack-operator-7847b5...	fleet-management	1	N/A	N/A	2	ReplicaSet	4d19h	Running
obs-kube-state-metrics-5f4cc9b856-5fd2c	fleet-management	1	N/A	N/A	1	ReplicaSet	4d19h	Running
obs-prometheus-node-exporter-tp7x	fleet-management	1	N/A	N/A	1	DaemonSet	4d19h	Running

FIGURE 4.4 – FreeLens — Vue Pods : 30 pods actifs dans le namespace fleet-management, tous en état Running

4.2.2 Vue Deployments

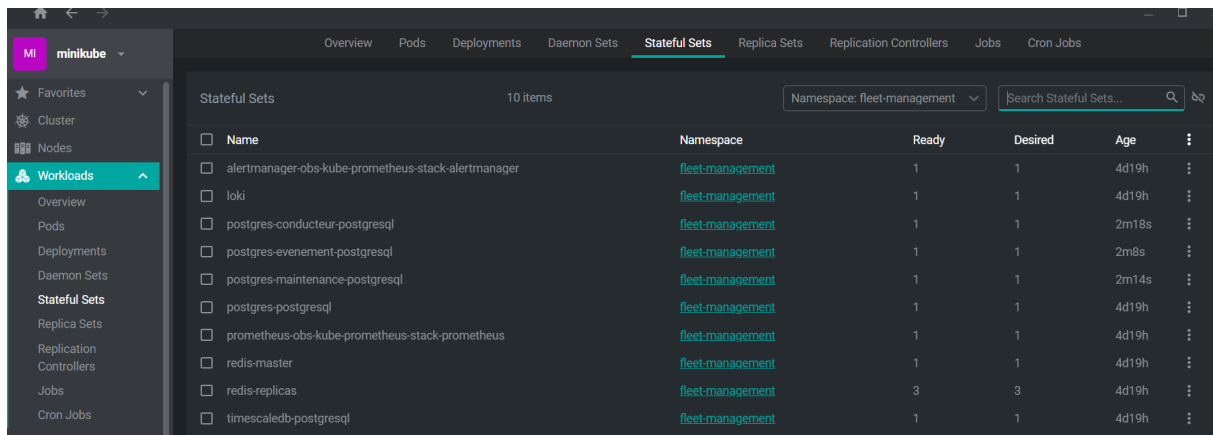


The screenshot shows the 'Deployments' view in the Kubernetes dashboard for the 'fleet-management' namespace. It displays a list of 10 deployments, all in a 'Progressing' state. The table includes columns for Name, Namespace, Ready, Desired, Updated, Available, Age, and Conditions. The deployments are managed by various controllers like ReplicaSet and StatefulSet.

Name	Namespace	Ready	Desired	Updated	Available	Age	Conditions
conducteur-service	fleet-management	2	2	2	2	4d19h	Available Progressing
evenement-service	fleet-management	2	2	2	2	4d19h	Available Progressing
localisation-service	fleet-management	2	2	2	2	4d19h	Available Progressing
maintenance-service	fleet-management	2	2	2	2	4d19h	Available Progressing
vehicule-service	fleet-management	2	2	2	2	4d19h	Available Progressing
obs-grafana	fleet-management	0	1	1	0	4d19h	
obs-kube-prometheus-stack-operator	fleet-management	1	1	1	1	4d19h	Available Progressing
obs-kube-state-metrics	fleet-management	1	1	1	1	4d19h	Available Progressing
otel-collector-opentelemetry-collector	fleet-management	1	1	1	1	4d19h	Available Progressing
strimzi-cluster-operator	fleet-management	1	1	1	1	4d19h	Available Progressing

FIGURE 4.5 – FreeLens — 10 Deployments dans le namespace fleet-management, tous disponibles

4.2.3 Vue Stateful Sets

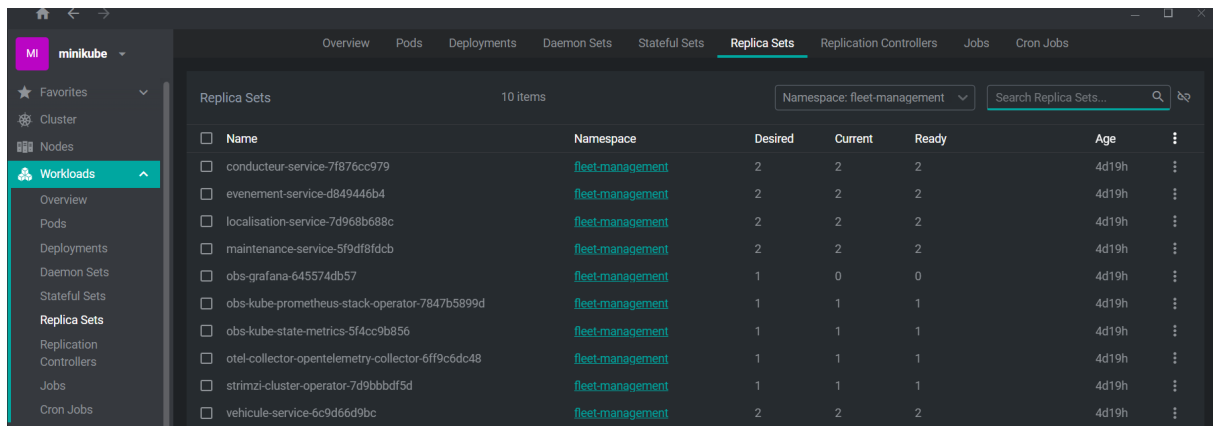


The screenshot shows the Kubernetes dashboard interface. The left sidebar has a menu with 'Workloads' selected. The main panel is titled 'Stateful Sets' and shows a list of 10 items in the 'fleet-management' namespace. The table lists the following Stateful Sets:

Name	Namespace	Ready	Desired	Age
alertmanager-obs-kube-prometheus-stack-alertmanager	fleet-management	1	1	4d19h
loki	fleet-management	1	1	4d19h
postgres-conducteur-postgresql	fleet-management	1	1	2m18s
postgres-evenement-postgresql	fleet-management	1	1	2m8s
postgres-maintenance-postgresql	fleet-management	1	1	2m14s
postgres-postgresql	fleet-management	1	1	4d19h
prometheus-obs-kube-prometheus-stack-prometheus	fleet-management	1	1	4d19h
redis-master	fleet-management	1	1	4d19h
redis-replicas	fleet-management	3	3	4d19h
timescaledb-postgresql	fleet-management	1	1	4d19h

FIGURE 4.6 – FreeLens — 10 Stateful Sets : bases de données et composants avec état persistant

4.2.4 Vue Replica Sets



The screenshot shows the Kubernetes dashboard interface. The left sidebar contains navigation links: Overview, Pods, Deployments, Daemon Sets, Stateful Sets, Replica Sets (selected), Replication Controllers, Jobs, and Cron Jobs. The main panel displays a table of Replica Sets in the 'fleet-management' namespace. The table has columns for Name, Namespace, Desired, Current, Ready, and Age. There are 10 items listed, all with an age of 4d19h.

Name	Namespace	Desired	Current	Ready	Age
conducteur-service-7f876cc979	fleet-management	2	2	2	4d19h
evenement-service-d849446b4	fleet-management	2	2	2	4d19h
localisation-service-7d968b688c	fleet-management	2	2	2	4d19h
maintenance-service-5f9df8fdb	fleet-management	2	2	2	4d19h
obs-grafana-645574db57	fleet-management	1	0	0	4d19h
obs-kube-prometheus-stack-operator-7847b5899d	fleet-management	1	1	1	4d19h
obs-kube-state-metrics-5f4cc9b856	fleet-management	1	1	1	4d19h
otel-collector-opentelemetry-collector-6ff9c6dc48	fleet-management	1	1	1	4d19h
strimzi-cluster-operator-7d9bbddf5d	fleet-management	1	1	1	4d19h
vehicule-service-6c9d66d9bc	fleet-management	2	2	2	4d19h

FIGURE 4.7 – FreeLens — 10 Replica Sets : gestion de la haute disponibilité

4.2.5 Résultat kubectl get all

```

Terminal X +
PS C:\Users\leiti\Systeme-de-Gestion-de-Flotte-de-Vehicules> kubectl get all -n fleet-management
NAME                                READY   STATUS              RESTARTS   AGE
pod/alertmanager-obs-kube-prometheus-stack-alertmanager-0  2/2     Running            2 (14m ago) 4d19h
pod/conducteur-service-7f876cc979-cxd6b                    1/1     Running            17 (62s ago) 4d19h
pod/conducteur-service-7f876cc979-xt8vk                    1/1     Running            17 (67s ago) 4d19h
pod/evenement-service-d849446b4-rhcnm                      1/1     Running            19 (94s ago) 4d19h
pod/evenement-service-d849446b4-sqpxg                     1/1     Running            19 (92s ago) 4d19h
pod/jaeger                                                  1/1     Running            1 (14m ago) 4d19h
pod/localisation-service-7d968b688c-m14mp                  1/1     Running            1 (14m ago) 4d19h
pod/localisation-service-7d968b688c-nb5cn                  1/1     Running            1 (14m ago) 4d19h
pod/loki-0                                                  1/1     Running            1 (14m ago) 4d19h
pod/loki-promtail-6dbck                                    1/1     Running            1 (14m ago) 4d19h
pod/maintenance-service-5f9df8fdb-79h6q                   1/1     Running            19 (96s ago) 4d19h
pod/maintenance-service-5f9df8fdb-lptfv                   1/1     Running            20 (87s ago) 4d19h
pod/obs-grafana-645574db57-fqhgq                           2/3     CrashLoopBackOff   20 (2s ago) 4d19h
pod/obs-kube-prometheus-stack-operator-7847b5899d-65v4g    1/1     Running            2 (3m22s ago) 4d19h
pod/obs-kube-state-metrics-5f4cc9b856-5fd2c                1/1     Running            1 (14m ago) 4d19h
pod/obs-prometheus-node-exporter-tpp7x                    1/1     Running            1 (14m ago) 4d19h
pod/otel-collector-opentelemetry-collector-6ff9c6dc48-7ct5z 1/1     Running            1 (14m ago) 4d18h
pod/postgres-conducteur-postgresql-0                      1/1     Running            0           39s
pod/postgres-evenement-postgresql-0                       1/1     Running            0           29s
pod/postgres-maintenance-postgresql-0                     1/1     Running            0           34s
pod/postgres-postgresql-0                                  1/1     Running            1 (14m ago) 4d19h
pod/prometheus-obs-kube-prometheus-stack-prometheus-0     2/2     Running            2 (14m ago) 4d19h
pod/redis-master-0                                         1/1     Running            1 (14m ago) 4d19h
pod/redis-replicas-0                                       1/1     Running            1 (14m ago) 4d19h
pod/redis-replicas-1                                       1/1     Running            1 (14m ago) 4d19h
pod/redis-replicas-2                                       1/1     Running            1 (14m ago) 4d19h
pod/strimzi-cluster-operator-7d9bbdbf5d-k5nfn              1/1     Running            1 (14m ago) 4d19h
pod/timescaledb-postgresql-0                              1/1     Running            1 (14m ago) 4d19h
pod/vehicule-service-6c9d66d9bc-hz4cv                     1/1     Running            19 (91s ago) 4d19h
pod/vehicule-service-6c9d66d9bc-pstjb                     1/1     Running            19 (93s ago) 4d19h

```

FIGURE 4.8 — Terminal FreeLens — kubectl get all -n fleet-management : vue complète de l'état du cluster

Chapitre 5

Semaine 3 — Débogage Kubernetes & Connectivité

L'étape la plus technique de la troisième semaine a été de résoudre une chaîne de crashes en cascade ("Bad Gateway") lors de l'intégration dans Minikube. L'infrastructure Kubernetes a nécessité des ajustements majeurs pour garantir la stabilité et la communication entre les microservices.

5.1 Stabilisation de Kafka (Transition vers KRaft)

Le service Véhicule crashait en boucle (*CrashLoopBackOff*) en essayant de joindre le broker Kafka. L'opérateur Strimzi récemment installé (v0.51+) ne supportait plus l'architecture historique basée sur Zookeeper que nous utilisions. **Solution** : Réécriture complète du manifeste `kafka.yaml` pour abandonner Zookeeper au profit du mode **KRaft**, via l'utilisation des ressources `KafkaNodePool` requises par les nouvelles versions de Kafka (4.1.0+).

5.2 Synchronisation des Ports Actifs

La communication entre Traefik, le Service K8s et le Pod était rompue. L'application Java écoutait sur le port 8080, tandis que l'exposition Docker et Kubernetes pointaient sur différents ports. **Solution** : Standardisation de l'ensemble de la chaîne applicative sur le port **4000** (Application properties, Dockerfile `EXPOSE`, Kubernetes `containerPort`).

5.3 Résilience et Sondes Kubernetes

Les pods comme le `conducteur-service` subissaient redémarrages intempestifs car Kubernetes considérait qu'ils ne répondaient pas (Probe failed 404). **Solution** : Les *Liveness Probes* ont été corrigées. Les services Python et Node.js ont été pourvus de routes spécifiques (`/health` et `/`) permettant à Minikube de valider la santé interne du pod et d'éviter les redémarrages en boucle.

5.4 Centralisation Ingress (Traefik)

Enfin, pour éviter les conflits de routage, la configuration Nginx a été supprimée du projet pour confier 100% du trafic externe entrant (via `api.fleet.local`) au reverse proxy **Traefik**, instaurant un point d'exposition propre et unique.

Chapitre 6

Bilan des livrables

secondary		
Dockerfiles multi-stage (5 services)	Complet	Build + Production, non-root
Docker Compose	Complet	Infrastructure + 5 microservices
Namespace Kubernetes	Complet	fleet-management
Secrets Kubernetes	Complet	DB, Kafka, Keycloak
Deployments K8s (5 services)	Complet	2 replicas, health checks
Services K8s (5 services)	Complet	ClusterIP
Ingress K8s	Complet	5 routes par service
PostgreSQL x4 (Helm Bitnami)	Complet	v18.5.6, une BD par service
TimescaleDB (Helm Bitnami)	Complet	v18.5.6, données GPS
Redis (Helm Bitnami)	Complet	v25.3.2
Kafka via Strimzi (Helm)	Complet	strimzi-operator v0.51.0
Prometheus + Grafana (Helm)	Complet	kube-prometheus-stack
Loki (Helm)	Complet	grafana/loki-stack v2.9.3
Jaeger	Complet	Pod kubectrl
OpenTelemetry Collector (Helm)	Complet	open-telemetry v0.147.0
CI/CD GitHub Actions	Complet	Build + Validate + Test
ADR (6 décisions)	Complet	ADR-001 à ADR-006

TABLE 6.1 – Bilan complet des livrables — Semaines 1 et 2

Chapitre 7

Conclusion

Les semaines 1 et 2 ont permis de poser des fondations solides et bien documentées pour le projet de Gestion de Flotte de Véhicules.

7.1 Ce que nous avons appris

Ce travail nous a permis de comprendre concrètement plusieurs concepts importants du développement cloud-native :

- **Le multi-stage build Docker** réduit significativement la taille des images de production et améliore la sécurité en excluant les outils de développement.
- **Kubernetes** permet de gérer la haute disponibilité automatiquement grâce aux replicas et aux health checks — si un pod tombe, Kubernetes en crée automatiquement un nouveau.
- **Helm** simplifie considérablement le déploiement d'applications complexes comme PostgreSQL ou Kafka, qui nécessiteraient sinon des dizaines de fichiers YAML.
- **Les ADR** sont un outil précieux pour documenter et justifier les choix techniques, facilitant la compréhension du projet par de nouveaux membres de l'équipe.
- **Strimzi** offre une meilleure intégration Kubernetes que les charts Helm traditionnels grâce au pattern opérateur — une approche à privilégier pour les composants stateful complexes.

7.2 Perspectives — Semaine 3

La semaine 3 sera consacrée au développement du premier microservice métier fonctionnel : **vehicule-service**. Les objectifs sont le développement de l'API REST CRUD complète, l'implémentation des événements Kafka (producer/consumer), la création des tests unitaires avec une couverture supérieure à 80%, et l'instrumentation OpenTelemetry pour les traces et métriques.

Rappel : Un système simple qui fonctionne vaut mieux qu'un système complexe qui ne fonctionne pas. L'approche itérative adoptée — commencer par l'infrastructure, puis enrichir service par service — nous permettra de livrer un système fonctionnel à chaque étape.